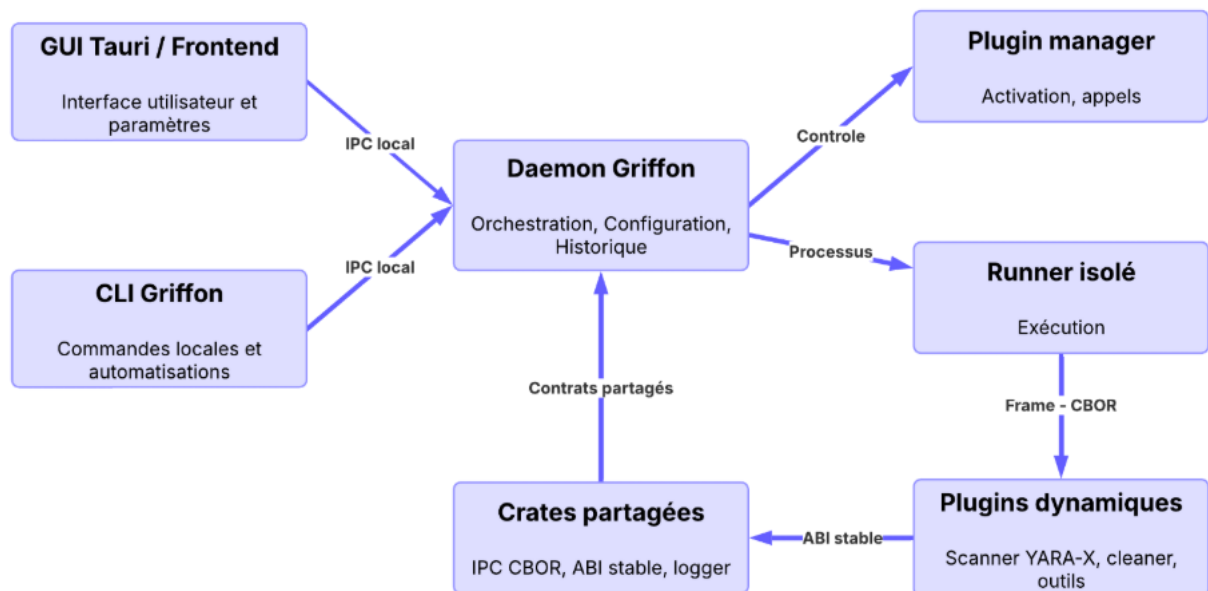


DOSSIER KPI 1

Évaluer et intégrer de nouvelles technologies

Projet Griffon — Plateforme de sécurité modulaire pour Linux

Veille technologique • POC • Benchmarks • Intégration • Documentation • Partage



Synthèse de l'architecture étudiée et mise en œuvre dans Griffon

Équipe projet : GriffonAV — 4 contributeurs principaux

Version du dossier : 1.1 — 15 juillet 2026

Fiche de contrôle du document

Élément	Valeur
Intitulé	KPI 1 — Evaluating and Integrating New Technologies (Technology Watch)
Projet	Griffon — plateforme de sécurité et d'outillage modulaire pour Linux
Périmètre	Architecture plugins, ABI, IPC, moteur de détection, GUI, benchmark et packaging
Période couverte	Fin 2025 à juillet 2026, complétée par les traces GitHub et les rapports de performance
Nature des preuves	Documentation interne, code et architecture, issues/PR GitHub, releases, rapports JSON, dashboard Grafana
Statut	Dossier prêt pour revue ; preuves de veille RSS et de participation communautaire externe intégrées.

Note méthodologique — Le dossier est une consolidation rétrospective fondée sur les documents et traces fournis. Aucun résultat de benchmark manquant n'a été inventé. Les valeurs issues du dashboard sont explicitement distinguées des mesures exactes présentes dans les fichiers JSON.

Résumé exécutif

Griffon est une plateforme de sécurité modulaire pour Linux, développée principalement en Rust. Le projet combine un daemon, une interface en ligne de commande, une application de bureau Tauri et un moteur d'extensions dynamiques. La proposition de valeur technique repose sur la possibilité d'écrire un outil de sécurité sous forme de plugin Rust, de décrire son interface dans un manifeste TOML, puis de laisser Griffon charger la bibliothèque, construire l'interface et assurer les communications.

Le KPI 1 est démontré par une démarche qui ne se limite pas à la lecture de documentation. L'équipe a identifié des problèmes structurants — absence de stabilité de l'ABI Rust native, besoin d'isoler l'exécution des plugins, volume des messages, intégration d'un moteur YARA dans un projet Rust, mesure des performances — puis a comparé plusieurs solutions, développé des preuves de concept, intégré les choix retenus et mesuré leurs effets.

Les décisions centrales sont l'adoption d'`abi\_stable` pour le contrat entre le host et les plugins, la construction d'un protocole IPC binaire Frame + CBOR avec une limite de payload, l'intégration de YARA-X plutôt que de la bibliothèque YARA C, l'utilisation de Tauri pour réunir frontend web et backend Rust, et la mise en place d'un pipeline de mesure JSON -> SQLite -> Grafana pour suivre les performances du Cleaner.

Les résultats fournis montrent que le Cleaner produit des métriques reproductibles et comparables entre versions et modes. Sur deux exécutions exactes en mode safe, le débit passe de 11,63 Go/s en v0.0.9 à 12,59 Go/s en v0.1.0, soit environ +8,3 %. Le dashboard agrégé suggère une amélioration moyenne plus importante, proche de 29 %, mais cette seconde valeur doit être confirmée par un export SQL complet avant d'être présentée comme mesure définitive.

Le dossier met également en évidence un retour d'expérience déterminant : un plugin renvoyait plus de 10 000 chemins de fichiers dans une réponse JSON, ce qui produisait un message trop volumineux et pouvait faire tomber le daemon au moment du transfert vers le frontend. La correction a consisté à limiter, trier et agréger les résultats, puis à renforcer le protocole avec une limite maximale de payload. Cet incident illustre la boucle complète attendue par le KPI : observation, analyse, expérimentation, correction, mesure et mise à jour de la documentation.

La couverture du KPI est élevée. La veille régulière est désormais matérialisée par des abonnements RSS ciblés — The Rust Foundation, The Hacker News et This Week in Rust — relayés dans le Discord de l'équipe via MonitoRSS. La participation communautaire est également prouvée par une publication sur r/rust sollicitant des retours sur l'auto-génération de la GUI, le daemon et l'architecture de plugins. Ces éléments complètent les preuves GitHub et montrent une démarche ouverte, même si l'animation communautaire doit rester continue dans le temps.

Table des matières

1. Cadre du KPI et objectifs du dossier
 2. Présentation de Griffon et problématique technique
 3. Organisation de la veille technologique
 4. Axe de veille n°1 — Architecture de plugins et ABI
 5. Axe de veille n°2 — IPC et sérialisation
 6. Axe de veille n°3 — Détection YARA-X
 7. Axe de veille n°4 — Application desktop Tauri
 8. Axe de veille n°5 — Mesure et visualisation des performances
 9. POC, expérimentations et intégrations concrètes
 10. Benchmark du plugin Cleaner
 11. Retour d'expérience : dépassement de taille des réponses
 12. Documentation, GitHub et partage des connaissances
 13. Synthèse d'impact et décisions
 14. Matrice de conformité au KPI
 15. Limites, risques et feuille de route
 16. Conclusion
- Annexes — preuves, journal de veille, données et bibliographie

1. Cadre du KPI et objectifs du dossier

Le KPI demande de démontrer une veille active et utile au projet, de produire des analyses comparatives, de réaliser une ou plusieurs expérimentations, d'intégrer au moins une nouvelle technologie dans le produit et d'en mesurer les effets. Il exige également que les résultats soient partagés dans l'équipe et, autant que possible, auprès de communautés techniques.

1.1 Interprétation opérationnelle

Attente du référentiel	Traduction pour Griffon	Preuve attendue
Veille active	Surveiller les technologies qui influencent directement le daemon, les plugins, le scanner, l'IPC et la GUI.	Notes datées, sources officielles, issues, décisions techniques.
Documentation et analyse	Comparer les alternatives selon sécurité, performance, coût d'intégration et maintenabilité.	Matrices de décision, rapports TECH/VEIL/PERF.
Application concrète	Tester puis intégrer une solution dans le dépôt.	POC, branches, PR, code fonctionnel, démonstration.
Mesure d'impact	Évaluer performance, stabilité, sécurité et temps de développement.	Rapports JSON, Grafana, incidents et corrections.
Partage et ouverture	Rendre la démarche vérifiable et réutilisable.	README, documentation, GitHub, reviews, changelog.

1.2 Question directrice

Problématique — Comment construire une plateforme Linux extensible par plugins Rust, performante et sûre, tout en maintenant un contrat stable entre composants, des communications maîtrisées et un processus de mesure reproductible ?

1.3 Critères de réussite retenus

- Un plugin peut être compilé et livré séparément du daemon.
- Une incompatibilité ou un message invalide doit produire une erreur contrôlée plutôt qu'un crash silencieux.
- Les appels sont corrélables, versionnés et limités en taille.
- Le scanner exploite un moteur de règles adapté à l'écosystème Rust.
- La GUI peut exploiter les fonctionnalités système sans réécrire toute l'application en toolkit natif.
- Les performances du Cleaner sont mesurées entre versions avec un format de rapport stable.
- Les décisions et méthodes sont documentées pour les autres développeurs.

2. Présentation de Griffon et problématique technique

Griffon est présenté dans la documentation comme un outil de sécurité rapide, modulaire et orienté Linux. Il fournit un daemon, une application de bureau, une CLI et un moteur de plugins. Les plugins intégrés incluent notamment un scanner YARA-X et un System Cleaner. [INT-INTRO] [INT-README]

2.1 Architecture du dépôt

```
griffon/
├── cli/           # interface en ligne de commande
├── daemon/        # daemon_core, daemon_runner, plugin_manager
├── gui/           # application Tauri et frontend web
├── plugins/       # scanner, cleaner, docker helper, template
├── shared/        # ipc_protocol, logger, plugin_interface
├── docs/          # documentation développeur
├── scripts/       # packaging .deb/.rpm
├── justfile       # automatisation des commandes
└── Cargo.toml    # workspace Rust
```

La séparation des modules répond à un besoin d'évolutivité : ajouter une fonctionnalité de sécurité ne doit pas imposer de modifier le cœur du daemon. Le modèle de plugin est volontairement réduit à deux artefacts : une bibliothèque ``.so`` contenant la logique et un manifeste ``.toml`` décrivant identité, état, disposition de l'interface et interactions. [INT-PLUGIN-ARCH]

2.2 Du prototype monolithique à la plateforme modulaire

Le guide de release distingue une version v0.2-alpha décrite comme un POC monolithique et une version v0.3 dotée d'un daemon modulaire, d'une GUI auto-générée et des plugins Scanner/Cleaner. Cette transition constitue le fil rouge du KPI : l'équipe n'a pas seulement ajouté une bibliothèque, elle a fait évoluer l'architecture globale après exploration et expérimentation. [INT-INSTALL]

2.3 Chronologie technique vérifiable

Date	Trace GitHub	Apport au KPI
27 nov. 2025	PR #90 — première version fonctionnelle du Plugin Manager	POC initial de découverte et gestion des plugins.
29 déc. 2025	PR #91 — IPC protocol V1	Premier contrat de communication entre manager et runner.
19 mars 2026	PR #118 — nouvelle architecture du daemon v0.1	Refonte expérimentale ; la PR indique explicitement que tout n'est pas encore fonctionnel.
1 avr. 2026	PR #122 — daemon de base fonctionnel	Validation de la nouvelle architecture.
11 mai–2 juin 2026	PR #149 — packaging et groupe Griffon	Passage du prototype à un produit installable sans sudo pour la CLI/GUI.
10–24 juin 2026	PR #175 — notifications et historique côté daemon	Instrumentation, traçabilité et contrôle des plugins.
26–30 juin 2026	PR #180 — notifications dans la GUI	Intégration bout en bout daemon -> GUI.
30 juin–5 juil. 2026	Rapports Cleaner v0.0.9 / v0.1.0	Campagne de mesure et visualisation Grafana.

3. Organisation de la veille technologique

3.1 Périmètre surveillé

- Langage et interopérabilité : ABI Rust, FFI, bibliothèques dynamiques, ``abi\_stable``.
- Architecture : daemon, runner, isolation de processus, contrats de plugins.
- Protocoles : framing, CBOR, corrélation de requêtes, contrôle de taille.
- Sécurité : YARA/YARA-X, règles de détection, gestion des permissions.
- Application desktop : Tauri, séparation frontend/backend et modèle de confiance.
- Performance : métriques, scénarios, variance, SQLite et Grafana.
- Déploiement : packages ``.deb``/``.rpm``, service systemd, droits de groupe.
- Organisation : branches, revues, conventions de commits et rapports techniques.

3.2 Sources utilisées

Catégorie	Exemples	Utilité
Sources officielles	Rust Reference, RFC 8949, docs `abi_stable`, YARA-X, Tauri, Grafana, systemd.	Établir les propriétés techniques et les limites des solutions.
Dépôts et issues	GitHub Griffon, PR, issues Cleaner/Scanner, historique des branches.	Prouver l'expérimentation et l'intégration réelle.
Documentation projet	README, guide plugins, IPC, Grafana, packaging, workflows.	Capitaliser les décisions et rendre le travail reproductible.
Mesures	Rapports JSON et dashboard Grafana.	Valider les impacts et détecter les régressions.
Communautés	Discord de l'équipe avec MonitoRSS, flux Rust Foundation / The Hacker News / This Week in Rust, publication r/rust et dépôt GitHub public.	Diffuser la veille, suivre les alertes et releases utiles, solliciter des avis externes et confronter les choix d'architecture.

3.3 Processus de veille et de décision

1. Formuler une question technique directement liée à une contrainte du projet.
2. Consulter en priorité les sources officielles et l'état des dépendances.
3. Lister au moins deux alternatives et leurs impacts sur Griffon.
4. Définir un POC limité dans le temps à partir du modèle interne.
5. Implémenter le test sur une branche ou dans un module isolé.
6. Mesurer les résultats et documenter les limites.
7. Prendre une décision : adopter, rejeter, différer ou lancer une expérimentation complémentaire.
8. Créer une issue/PR et mettre à jour la documentation ou l'architecture.

Le modèle POC interne impose de documenter le contexte, l'objectif, le temps alloué, les résultats et le suivi. Le modèle de rapport de performance exige quant à lui une question mesurable, des visualisations, une conclusion et des actions. [INT-POC] [INT-PERF-TPL]

3.4 Journal de veille consolidé

Sujet	Question	Expérience / preuve	Décision
ABI et plugins	Comment charger des plugins Rust sans dépendre de l'ABI native instable ?	Plugin Manager, interface partagée, `cdylib`, `abi_stable`, PR #90.	Adopter `abi_stable` ; garder l'ABI C brute comme alternative de secours.
IPC	Comment délimiter et corréler des messages sur un flux local ?	Frame 12 octets, CBOR, request_id, handshake, PR #91.	Adopter le protocole v1 avec limite de 1 Mo.
Détection	YARA C ou YARA-X ?	Étude FFI C vs crate Rust ; intégration Scanner.	Adopter YARA-X, surveiller les versions et changements de dépendances.
GUI	Comment réutiliser les compétences web tout en conservant un backend Rust ?	Prototype et intégration Tauri.	Adopter Tauri ; documenter strictement les frontières de confiance.

Sujet	Question	Expérience / preuve	Décision
Performance	Comment comparer le Cleaner entre versions ?	Rapports JSON, SQLite, Grafana, campagnes v0.0.9/v0.1.0.	Conserver l'instrumentation et définir des scénarios reproductibles.
Isolation future	Une sandbox WebAssembly apporterait-elle un bénéfice ?	Analyse documentaire du Component Model.	Option différée : intéressante mais trop coûteuse pour la version actuelle.
Veille partagée et communauté	Comment maintenir une veille régulière et confronter les choix Griffon à des développeurs externes ?	Flux RSS relayés dans Discord via MonitoRSS ; suivi d'avis RUSTSEC, releases Rust/YARA-X ; post r/rust sur la GUI auto-générée.	Conserver ce canal, archiver les éléments ayant un impact et renouveler les échanges communautaires à chaque décision structurante.

4. Axe de veille n°1 — Architecture de plugins et ABI

4.1 Problème identifié

L'ABI Rust native n'offre pas de garantie de stabilité. La Rust Reference recommande de spécifier explicitement une ABI externe et indique que l'ABI Rust ne fournit pas de stabilité. Un plugin compilé avec une version différente du compilateur ou avec une disposition de types différente peut donc devenir incompatible. [EXT-RUST-ABI]

Griffon avait besoin de plugins compilables indépendamment, capables d'exposer des métadonnées et une fonction de traitement sans lier statiquement tous les outils au daemon. Cela excluait une exposition naïve de ``String``, ``Vec`` ou ``Result`` à travers la frontière binaire.

4.2 Solutions comparées

Solution	Avantages	Inconvénients	Adéquation Griffon
Rust ABI native	Code idiomatique et aucun adaptateur visible.	Aucune garantie de stabilité ; risque élevé à chaque changement de toolchain.	Rejetée pour la frontière de plugin.
ABI C manuelle	Contrat largement supporté ; interopérabilité multilangage.	Conversions et ownership manuels ; surface FFI plus complexe ; risque d'erreurs mémoire.	Alternative viable mais plus coûteuse pour une équipe Rust.
<code>`abi_stable`</code>	Types ABI-safe, modules racine, contrôle de types au chargement, logique Rust-to-Rust.	Types spécifiques, apprentissage, pas de déchargement transparent, API à discipliner.	Retenue pour la version actuelle.
WebAssembly Component Model	Isolation, portabilité, contrats WIT, potentiel multilangage.	Runtime et toolchain supplémentaires, accès système à redéfinir, migration des plugins.	Perspective, non retenue immédiatement.

4.3 Choix retenu : ``abi_stable``

La documentation officielle de ``abi_stable`` décrit une bibliothèque Rust-to-Rust destinée aux bibliothèques chargées à l'exécution et proposant une vérification de types au chargement. Ce positionnement correspond directement au besoin de Griffon. [EXT-ABI-STABLE]

```
#[repr(C)]
#[derive(StableAbi)]
#[sabi(kind(Prefix(prefix_ref = PluginRef)))]
pub struct PluginI {
    pub init: extern "C" fn() -> RResult<RVec<Tuple2<RString, RString>>, RString>,
    pub handle_message: extern "C" fn(RString) -> RString,
}
```

Les types échangés sont `RString`, `RVec` et `RResult`. Le plugin exporte un module racine et expose deux fonctions : `init()` pour l'identité et les capacités, puis `handle\_message()` pour le dispatch des commandes. [INT-RUST-BACKEND] [INT-PLUGIN-GUIDE]

4.4 Architecture à deux fichiers

Artefact	Responsabilité	Conséquence
.so	Logique native du plugin, métadonnées et fonctions exportées.	Peut évoluer indépendamment du frontend.
.toml	Identité, store, layout, composants et interactions.	L'interface est construite sans écrire une page React spécifique.
.zip de livraison	Contient la bibliothèque et le manifeste.	Installation simplifiée depuis les paramètres Griffon.

L'interface TOML décrit un store réactif, les sections, les tableaux, boutons, champs et actions. Les interactions peuvent modifier le store ou appeler une fonction Rust, puis stocker la réponse JSON. [INT-UI] [INT-INTERACTIONS]

4.5 Décision et limites

Décision — `abi\_stable` est retenu car il réduit le coût de l'ABI C tout en permettant un chargement dynamique Rust-to-Rust avec vérification au chargement.

- Le contrat doit rester minimal et versionné.
- Les types Rust standard ne doivent pas traverser la frontière ABI.
- Un panic ne doit pas sortir de `init()` ou `handle\_message()`.
- Les fonctions longues doivent être déportées vers un thread ou un runner afin de ne pas bloquer l'appel.
- Le déchargement à chaud et la sandbox complète ne sont pas couverts par ce choix.

5. Axe de veille n°2 — IPC et sérialisation

5.1 Besoin

Un flux Unix ne contient pas naturellement de frontières de messages. Griffon doit donc savoir où commence et où se termine chaque message, vérifier que les deux composants utilisent la même version, associer une réponse à une requête et refuser les tailles anormales.

5.2 Comparaison des formats

Format	Points forts	Points faibles	Usage retenu
JSON	Lisible, facile à diagnostiquer, très utilisé dans la GUI et les manifests.	Plus verbeux ; chaîne souvent utilisée comme enveloppe supplémentaire.	Configuration, messages applicatifs lisibles et réponses de plugins.

Format	Points forts	Points faibles	Usage retenu
CBOR	Binaire compact, modèle proche de JSON, standard RFC, compatible Serde.	Moins lisible sans outil ; schémas à synchroniser.	Payload du protocole Plugin Manager <-> Runner.
MessagePack	Compact et largement supporté.	Une dépendance/standard supplémentaire sans avantage décisif démontré.	Non retenu.
Bincode	Très direct dans un environnement Rust.	Couplage fort aux représentations et à l'évolution des structures.	Non retenu pour un protocole versionné.

Le RFC 8949 définit CBOR comme un format visant notamment une faible taille de message, une faible complexité et l'extensibilité. Ces objectifs correspondent à un canal local de contrôle. [EXT-CBOR]

5.3 Protocole Griffon v1

Champ	Taille	Rôle
magic	2 octets	Détecte un flux désynchronisé ; valeur 0xBEEF.
version	1 octet	Refuse un protocole incompatible ; valeur actuelle 1.
type	1 octet	Hello, HelloOk, Call, Result, Heartbeat ou Error.
request_id	4 octets	Corrèle plusieurs appels en vol.
payload_len	4 octets	Détermine la taille du payload.
payload	N octets	CBOR pour les messages typés ; vide pour Hello/Heartbeat.

Le parseur vérifie le magic, la version, le type et la taille. `MAX\_PAYLOAD` est fixé à 1 Mo, ce qui évite une allocation arbitraire en cas de flux corrompu ou d'entrée abusive. [INT-IPC]

5.4 Flux fonctionnels

```
Handshake
Plugin Manager -> Hello (request_id = 0)
Runner        -> HelloOk { name, functions }

Appel
Plugin Manager -> Call { request_id = 42, fn_name, args }
Runner        -> Result { request_id = 42, ok, output }
               ou Error { request_id = 42, code, message }
```

5.5 Analyse critique

- La limite de 1 Mo protège le transport, mais ne remplace pas la pagination et l'agrégation métier.
- Le champ de version doit être accompagné d'une politique explicite de compatibilité lors du passage à v2.
- `ResultPayload.output` reste une chaîne qui peut elle-même contenir du JSON : cette double couche est pratique mais moins typée.
- Le heartbeat permet de détecter un runner bloqué ou mort, mais nécessite des délais et actions de récupération clairement définis.
- Les codes d'erreur doivent être centralisés pour éviter des conventions divergentes entre plugins.

6. Axe de veille n°3 — Détection YARA-X

6.1 Contexte

YARA permet de classer et identifier des échantillons à partir de motifs textuels ou binaires et de conditions. La documentation Griffon indique que le choix initial envisagé était la bibliothèque YARA en C, puis que l'équipe a privilégié YARA-X, réécriture officielle en Rust maintenue dans l'écosystème VirusTotal. [INT-YARA]

La documentation officielle présente YARA-X comme une nouvelle incarnation de YARA visant notamment davantage de vitesse, de sécurité et de facilité d'utilisation. L'API Rust est native puisque le moteur est écrit en Rust. YARA-X 1.0 a été déclaré stable en juin 2025. [EXT-YARAX-START] [EXT-YARAX-RUST] [EXT-YARAX-STABLE]

6.2 Comparaison

Critère	YARA C	YARA-X	Conséquence Griffon
Intégration Rust	FFI, wrappers, build natif et conversions.	Crate et API Rust directes.	Réduction du code de glue.
Sécurité mémoire	Dépend du code C et des wrappers.	Implémentation Rust pour le cœur du moteur.	Meilleure cohérence avec la stack.
Maturité	Très forte ancienneté et écosystème établi.	Plus récent ; stabilisé en 1.0 mais évolutif.	Surveillance de versions indispensable.
Compatibilité de règles	Référence historique.	Compatibilité élevée avec différences documentées.	Tester le corpus de règles avant mise à jour.
Maintenance	Dépendances C et packaging système.	Dépendances Cargo mais builds parfois sensibles aux versions.	Choix favorable, avec pinning et CI.

6.3 Expérience et intégration

Le Scanner Griffon charge les règles, parcourt fichiers et dossiers, gère les archives, la quarantaine et le multithreading. L'issue #130 matérialise ce périmètre. L'équipe a également rencontré une incompatibilité de compilation entre versions de `yara-x` et de ses structures de build, ce qui illustre le risque réel d'une technologie récente : l'intégration est meilleure, mais la gestion des versions devient un élément du choix.

Décision — YARA-X est retenu pour l'intégration native Rust et la sécurité de la chaîne, avec verrouillage des versions, tests de compilation et validation d'un corpus de règles.

6.4 Règles et sources

La documentation actuelle explique la structure des règles et mentionne la nécessité de s'appuyer sur des chercheurs spécialisés, mais les liens de provenance des règles restent à compléter. Pour une plateforme de sécurité, cette lacune doit être corrigée : chaque source de règles doit être identifiée, versionnée, licenciée et évaluée pour les faux positifs.

- Conserver une liste des dépôts et licences des règles.
- Enregistrer la date de mise à jour et le hash du corpus.
- Exécuter un jeu de tests positifs et négatifs avant publication.
- Documenter les différences YARA/YARA-X utilisées par les règles.
- Prévoir une stratégie de retour arrière en cas de régression.

7. Axe de veille n°4 — Application desktop Tauri

7.1 Besoin

L'équipe devait fournir une interface moderne tout en conservant l'accès aux fonctions système, aux fichiers locaux et au daemon Rust. Réécrire l'ensemble de l'interface avec un toolkit natif aurait augmenté le temps de développement et réduit la réutilisation des compétences TypeScript/React.

7.2 Adéquation de Tauri

Tauri combine un cœur Rust et une interface HTML rendue dans une WebView système. La communication frontend -> Rust repose sur le passage de messages et des commandes. Sa documentation insiste aussi sur la frontière de confiance entre le code Rust disposant de capacités système et le frontend WebView. [EXT-TAURI-ARCH] [EXT-TAURI-SEC] [EXT-TAURI-CALL]

Option	Atout principal	Coût / risque	Décision
Tauri	Cœur Rust, frontend web, commandes typées, utilisation de la WebView système.	Permissions et commandes à limiter ; dépendances WebKitGTK sous Linux.	Retenue.
Electron	Écosystème web complet et moteur homogène.	Runtime distinct et architecture moins alignée avec le backend Rust.	Non retenue dans la version actuelle.
Toolkit Rust/GTK	Interface native et intégration Linux.	Courbe UI différente, réécriture et productivité initiale plus faible pour l'équipe.	Non retenu.
GUI web distante	Déploiement simple du frontend.	Nécessite une API locale/réseau et modifie le modèle de menace.	Non retenue.

7.3 Auto-génération de l'interface plugin

Le manifeste TOML est une évolution propre à Griffon : il déclare l'identité, le store, les sections, les éléments visuels et les interactions. Un bouton peut déclencher une action, une interaction appelle `handle_message()`, puis le résultat JSON met à jour le store et l'interface se rafraîchit. [INT-UI] [INT-INTERACTIONS]

Cette abstraction réduit le coût de création d'un plugin et évite de modifier la GUI centrale pour chaque outil. Elle introduit toutefois un nouveau risque : les incohérences entre UUID, noms de fonctions, onglets et interactions peuvent ne se révéler qu'au clic utilisateur. Le guide de packaging recommande donc une vérification manuelle ; un validateur de manifeste est identifié comme amélioration prioritaire. [INT-PACKAGING]

7.4 Incohérence documentaire détectée

Point de qualité — Le README récent décrit une GUI Tauri v2 + Vite, tandis qu'une page d'introduction plus ancienne mentionne Tauri + Next.js. Le dossier retient la stack du README et recommande d'harmoniser la documentation pour éviter une preuve contradictoire.

8. Axe de veille n°5 — Mesure et visualisation des performances

8.1 Objectif

Le projet ne considère pas une optimisation comme valide uniquement parce que l'application semble plus rapide. Le Cleaner génère des rapports structurés et Grafana est utilisé comme couche de visualisation au-

dessus d'une base SQLite. L'objectif est de comparer les versions, détecter les régressions et produire des graphiques réutilisables. [INT-GRAFANA]

8.2 Chaîne de données

```

Exécution du Cleaner
↓
Rapport JSON versionné et daté
↓
Import dans une base SQLite
↓
Requêtes SQL de comparaison
↓
Dashboard Grafana
↓
Rapport PERF et décision de release
  
```

8.3 Schéma de métriques

Champ	Signification	Usage
scenario	Jeu d'essai, par ex. `basic_clean`.	Comparer des exécutions équivalentes.
version	Version du plugin.	Mesurer une amélioration ou régression.
mode	`safe` ou `aggressive`.	Comparer le compromis profondeur/vitesse.
bytes_freed_total	Volume identifié ou libéré selon le mode.	Mesure de résultat ; sémantique à clarifier.
files_scanned_total	Nombre de fichiers visités.	Normaliser la charge.
files_cleaned_total	Nombre de fichiers supprimés.	Mesurer l'action réelle.
run_duration_seconds	Temps total.	Calcul de performance.
bytes_freed_per_second	Débit calculé.	Comparaison synthétique.
errors_by_type	Erreurs regroupées, ex. permission denied.	Qualité et robustesse.

8.4 Pourquoi Grafana

Dans Griffon, Grafana n'est pas utilisé comme plateforme d'alerte temps réel. Il sert de couche de reporting SQL pour organiser des dashboards et des panels comparatifs. La documentation officielle de Grafana confirme que les panels associent requêtes et visualisations, et que le modèle JSON du dashboard stocke panels, variables et paramètres. [EXT-GRAFANA-DASH]

- Dashboards réutilisables entre releases.
- Comparaison visuelle de plusieurs versions et modes.
- Requêtes SQL explicites et auditable.
- Séparation entre collecte des données et présentation.
- Export de graphiques pour les rapports techniques.

9. POC, expérimentations et intégrations concrètes

9.1 POC n°1 — Plugin Manager et ABI stable

Rubrique	Contenu
Contexte	Le prototype monolithique limitait l'ajout indépendant de fonctionnalités.

Rubrique	Contenu
Objectif fonctionnel	Charger une ``.so``, appeler ``init()`, découvrir les fonctions et transmettre une commande.
Question	``abi_stable`` permet-il un contrat suffisamment simple pour les plugins Griffon ?
Preuve	PR #90, interface partagée, plugin template et documentation backend.
Résultat	Première version fonctionnelle du Plugin Manager ; modèle repris par les plugins intégrés.
Décision	Adoption dans l'architecture de production.
Suivi	Packaging ``.so`` + ``.toml``, installation depuis la GUI, suppression et refresh.

9.2 POC n°2 — Protocole Frame + CBOR

Rubrique	Contenu
Contexte	Le runner et le Plugin Manager ont besoin d'un transport local fiable.
Objectif fonctionnel	Échanger Hello/Call/Result/Error sur un flux Unix avec corrélation.
Question	Un framing fixe et CBOR suffisent-ils pour des messages typés et bornés ?
Preuve	PR #91 et documentation IPC complète.
Résultat	Header 12 octets, version, request_id, MAX_PAYLOAD 1 Mo et handshake.
Décision	Adoption du protocole v1 dans le périmètre Plugin Manager.
Suivi	Définir politique v2, codes d'erreur et tests de fuzzing.

9.3 Expérimentation n°3 — Migration YARA vers YARA-X

Rubrique	Contenu
Contexte	La bibliothèque YARA C introduit FFI et packaging natif.
Objectif	Intégrer un moteur de règles dans le scanner Rust.
Question	YARA-X simplifie-t-il suffisamment l'intégration tout en conservant les capacités utiles ?
Résultat	Scanner YARA-X intégré ; API Rust directe ; règles et quarantaine.
Difficulté	Erreur de build lors d'un décalage entre versions de dépendances.
Décision	Adoption avec versions verrouillées et CI.

9.4 Intégration produit

- Le dépôt contient des plugins réels et un plugin template.
- Le manifeste TOML génère le store et l'interface.
- La GUI et la CLI appellent le daemon.
- Le daemon contrôle l'état, les notifications et l'historique des plugins.
- Les plugins peuvent être packagés et chargés depuis les paramètres.
- Les packages ``.deb`` et ``.rpm`` installent le daemon et le service systemd.
- La documentation propose un parcours utilisateur et un parcours contributeur.

10. Benchmark du plugin Cleaner

10.1 Protocole disponible

Les rapports fournis portent sur le scénario `basic\_clean`, les versions v0.0.9 et v0.1.0, et les modes safe/aggressive. Les fichiers sont nommés avec la date, le scénario et le mode. Cette convention facilite l'import et la traçabilité.

10.2 Échantillons exacts fournis

Date	Version	Mode	Scannés	Nettoyés	Volume	Durée	Débit	Perm.
2026-07-03	v0.0.9	aggressive	107 563	6 231	15,16 Go	1,460 s	10,385 Go/s	22
2026-07-03	v0.0.9	safe	107 563	0	13,50 Go	1,279 s	10,557 Go/s	19
2026-07-04	v0.0.9	aggressive	108 793	7 093	15,63 Go	1,489 s	10,500 Go/s	20
2026-07-04	v0.0.9	safe	108 793	0	13,52 Go	1,163 s	11,628 Go/s	15
2026-07-05	v0.1.0	safe	110 388	0	14,29 Go	1,135 s	12,591 Go/s	15

Conversion utilisée pour la lecture : 1 Go = 10⁹ octets, cohérente avec le libellé `gb\_per\_second` du dashboard. Les valeurs source restent disponibles en octets dans les JSON.

10.3 Comparaison exacte de deux runs safe

Le run safe v0.0.9 du 4 juillet atteint 11,628 Go/s. Le run safe v0.1.0 du 5 juillet atteint 12,591 Go/s. L'amélioration calculée sur ces deux exécutions est :

$$(12,591 - 11,628) / 11,628 \times 100 = 8,28 \%$$

Résultat exact — Sur les deux exécutions safe explicitement fournies, v0.1.0 est environ 8,3 % plus rapide que v0.0.9. Cette comparaison reste un résultat unitaire et non une moyenne statistique.

10.4 Dashboard agrégé



Figure — Dashboard Grafana : débit par version, débit par mode et variance

La lecture visuelle du panel « Scan Speed per version » situe v0.0.9 autour de 9,1 Go/s et v0.1.0 autour de 11,7 Go/s. Cela suggère une amélioration moyenne proche de 29 %. Le dashboard indique également une variance très faible en v0.0.9 (0,0000168) et plus élevée en v0.1.0 (0,122).

Précaution — La valeur de +29 % est une estimation issue de la hauteur des barres. Elle ne doit devenir un chiffre officiel qu’après export de la requête SQL ou de la table agrégée utilisée par Grafana.

10.5 Interprétation

- Le débit progresse malgré une légère hausse du nombre de fichiers scannés dans l’exemple v0.1.0.
- Le mode aggressive nettoie plusieurs milliers de fichiers et rencontre davantage d’erreurs de permission dans les runs disponibles.
- Le mode safe retourne `files_cleaned_total = 0``, ce qui doit être expliqué comme dry-run ou absence de suppression afin d’éviter une ambiguïté.
- La variance v0.1.0 plus élevée peut indiquer un effet du mode, de l’état du cache ou d’un environnement moins stable.
- Les résultats prouvent l’existence d’une instrumentation, mais le protocole matériel et logiciel doit être fixé pour une comparaison scientifique.

10.6 Renforcement recommandé du benchmark

1. Utiliser la même machine, le même jeu de données et la même configuration de puissance.
2. Exécuter au minimum 10 répétitions par version et par mode.
3. Enregistrer CPU, RAM, noyau, disque, état du cache et commit Git.
4. Calculer moyenne, médiane, écart-type, minimum, maximum et intervalle de confiance.

5. Séparer le temps de parcours, le temps de classification et le temps de suppression.
6. Verrouiller la signification de ``bytes_freed_total`` pour safe et aggressive.
7. Exporter la requête Grafana/SQLite utilisée pour chaque chiffre présenté.

11. Retour d'expérience : dépassement de taille des réponses

11.1 Incident

Pendant le développement du Cleaner, le plugin renvoyait plus de 10 000 noms de fichiers. La réponse JSON devenait très volumineuse. Le daemon recevait cette chaîne puis tentait de la transférer au frontend, ce qui entraînait une consommation excessive et pouvait faire tomber le daemon.

11.2 Cause racine

Niveau	Cause	Effet
Métier	Le plugin exposait chaque chemin individuel, sans limite ni agrégation.	Croissance linéaire du nombre d'objets et de chaînes.
Sérialisation	Le JSON répétait les clés et les préfixes de chemins.	Payload plus volumineux que l'information utile.
Transport	Aucune politique métier de pagination sur le résultat.	Un seul message devait contenir la totalité.
Frontend	Le rendu d'une liste massive n'était pas la meilleure représentation.	Coût mémoire et UX dégradée.

11.3 Correction appliquée

- Sélection des fichiers les plus volumineux au lieu de retourner tous les chemins.
- Agrégation des résultats par racine ou catégorie.
- Conservation de métriques synthétiques utiles au dashboard.
- Limitation du payload dans le protocole IPC.
- Découplage entre la vue de synthèse et les actions ciblées de suppression.

11.4 Enseignements

Leçon principale — Choisir CBOR ou un format plus compact ne résout pas un modèle de données non borné. L'optimisation la plus importante a été de réduire et structurer l'information avant sérialisation.

Cet incident est particulièrement pertinent pour le KPI : il montre que l'équipe a observé un comportement réel, identifié une cause située à plusieurs couches, modifié l'architecture de réponse et ajouté des protections. Il justifie également la présence de ``MAX_PAYLOAD`` dans le protocole.

12. Documentation, GitHub et partage des connaissances

12.1 Documentation produite

Document	Apport
README	Installation, exécution, architecture du dépôt et commandes principales.
Introduction / Installation	Positionnement, versions, compilation et releases.
Plugin Architecture	Modèle <code>`so`</code> + <code>`toml`</code> et boucle action -> interaction -> store.

Document	Apport
UI Manifest & Store	Schéma déclaratif de l'interface et du state.
Interactions	Connexion entre actions UI et fonctions Rust.
Rust Backend	ABI, signatures, registry de commandes et gestion d'état.
Packaging & Loading	Compilation, cohérence du manifeste et installation ZIP.
IPC Protocol	Frame, CBOR, types, limites et erreurs.
YARA	Raisons du passage de YARA C à YARA-X.
Grafana	Usage des dashboards et requêtes SQL.
POC/PERF templates	Méthode standardisée pour expérimenter et mesurer.
Workflow Git	Branches, commits conventionnels et revues de PR.
Captures RSS / Discord / Reddit	Preuves datées de veille régulière, diffusion au sein de l'équipe et sollicitation de retours externes.

12.2 Organisation GitHub

Les règles de branche interdisent les commits directs sur `main`, recommandent une branche par fonctionnalité et imposent une revue avant fusion vers `develop`. Les pull requests vers `main` doivent être préparées collectivement. Les commits suivent une convention de type `feat`, `fix`, `perf`, `docs`, `build`, etc. [INT-BRANCH] [INT-PR] [INT-COMMITS]

12.3 Exemples de preuves de partage et d'intégration

Preuve	Objet	Valeur pour le KPI
PR #90	Plugin Manager fonctionnel.	POC intégré et partagé dans la branche commune.
PR #91	Protocole IPC v1.	Formalisation d'un contrat partagé.
PR #149	Packaging, groupe Griffon et suppression du besoin de sudo pour CLI/GUI.	Industrialisation et prise en compte de la sécurité opérationnelle.
PR #175	Notifications et historique côté daemon.	Traçabilité et qualité produit.
PR #180	Notifications dans la GUI.	Intégration bout en bout et collaboration frontend/backend.
Issues #80–89	Décomposition du Cleaner : trait, registry, report, filtres, parcours, statistiques, erreurs.	Planification fine et capitalisation des choix.
Issue #130	Scanner YARA-X, archives, règles, quarantaine, multithreading.	Périmètre technique explicite.
Issue #134	Connexion Cleaner <-> frontend.	Validation de l'intégration utilisateur.

12.4 Ouverture et communautés

Le dépôt est public, sous licence Apache-2.0, et la documentation invite aux retours, revues de code et contributions. En complément de cette ouverture passive, l'équipe a mis en place une veille partagée dans Discord et a directement sollicité la communauté Rust sur Reddit. Les preuves ci-dessous montrent à la fois la régularité de la veille et une première démarche de dialogue externe.

Preuve ajoutée — La participation communautaire est matérialisée par un post sur r/rust présentant Griffon et sollicitant des retours sur l’auto-génération de la GUI. La veille régulière est prouvée par des flux RSS ciblés relayés dans Discord via MonitoRSS. La démarche est donc couverte, avec une recommandation de poursuivre les échanges et d’archiver leurs impacts sur les décisions.

12.4.1 Veille RSS partagée dans Discord

L’équipe a configuré un ensemble de flux RSS ciblés, puis les a relayés dans un canal Discord au moyen de MonitoRSS. Les abonnements visibles couvrent The Rust Foundation, The Hacker News et This Week in Rust. La liste indique que This Week in Rust était suivi depuis le 17 octobre 2025 et que les deux autres sources ont été ajoutées le 9 avril 2026. Cette sélection associe actualité du langage, sécurité et écosystème open source.

☐	STATUS	TITLE	URL	ADDED ON	SHARED WITH ME
☐	✓	The Rust Foundation	https://rustfoundation.org/feed/	2026/04/09	Configure >
☐	✓	The Hacker News	https://feeds.feedburner.com/Th...	2026/04/09	Configure >
☐	✓	the week-in-rust	https://this-week-in-rust.org/rss.xml	2025/10/17	Configure >

Figure 8 — Liste des flux RSS suivis pour la veille Griffon (capture du 15 juillet 2026).

Le relais dans Discord rend l’information immédiatement visible par toute l’équipe sans imposer à chaque membre de consulter séparément les sites. Les messages de décembre 2025 montrent notamment un avis RUSTSEC, une actualité sur l’intégration durable de Rust dans le noyau Linux et l’annonce de Rust 1.92.0. Ces éléments nourrissent la veille sur la sécurité de la supply chain, l’évolution du langage et son adoption dans l’écosystème Linux.

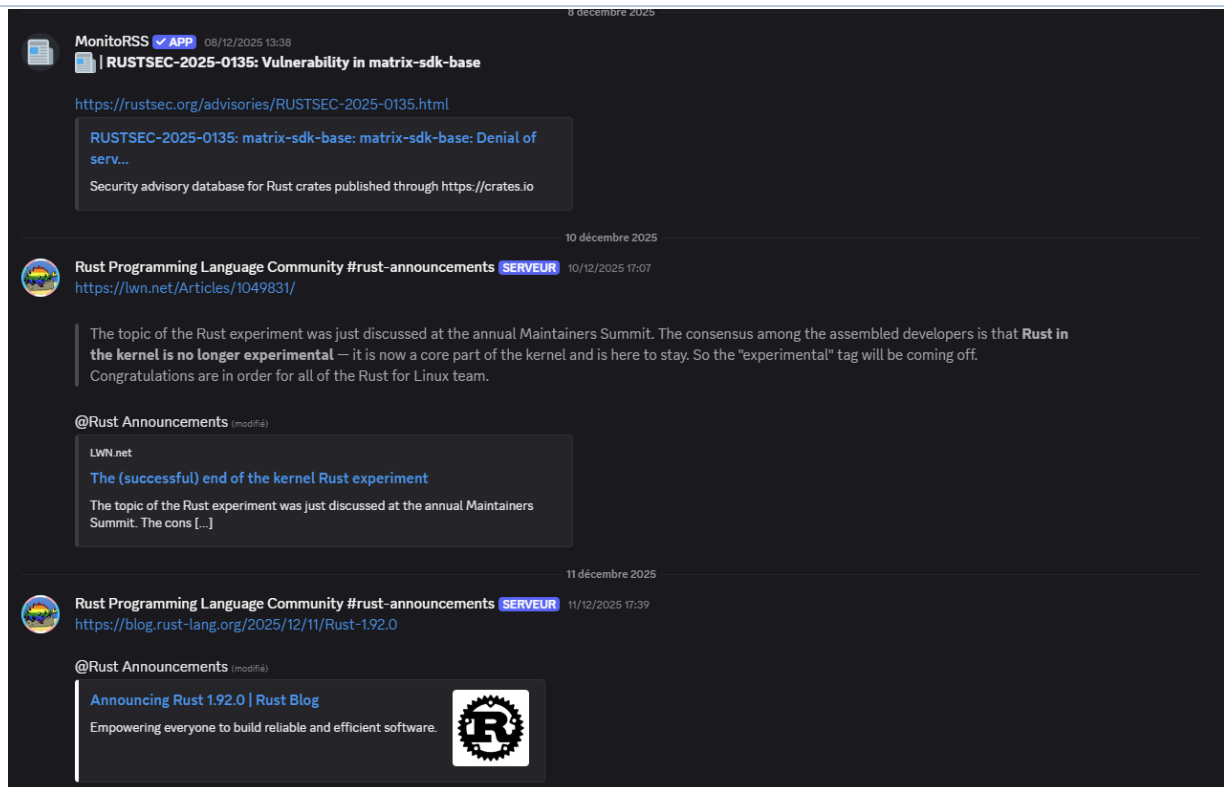


Figure 9 — Exemples de veille Rust et sécurité relayés dans Discord en décembre 2025.

Les messages de janvier 2026 montrent la continuité du dispositif : publication de This Week in Rust 633, release YARA-X v1.11.0 et nouvel avis RUSTSEC. La release YARA-X est directement pertinente pour Griffon, car le Scanner dépend de cette technologie. Le suivi ne porte donc pas uniquement sur des tendances générales : il couvre aussi les composants réellement intégrés au projet et les vulnérabilités susceptibles d'affecter les dépendances Rust.

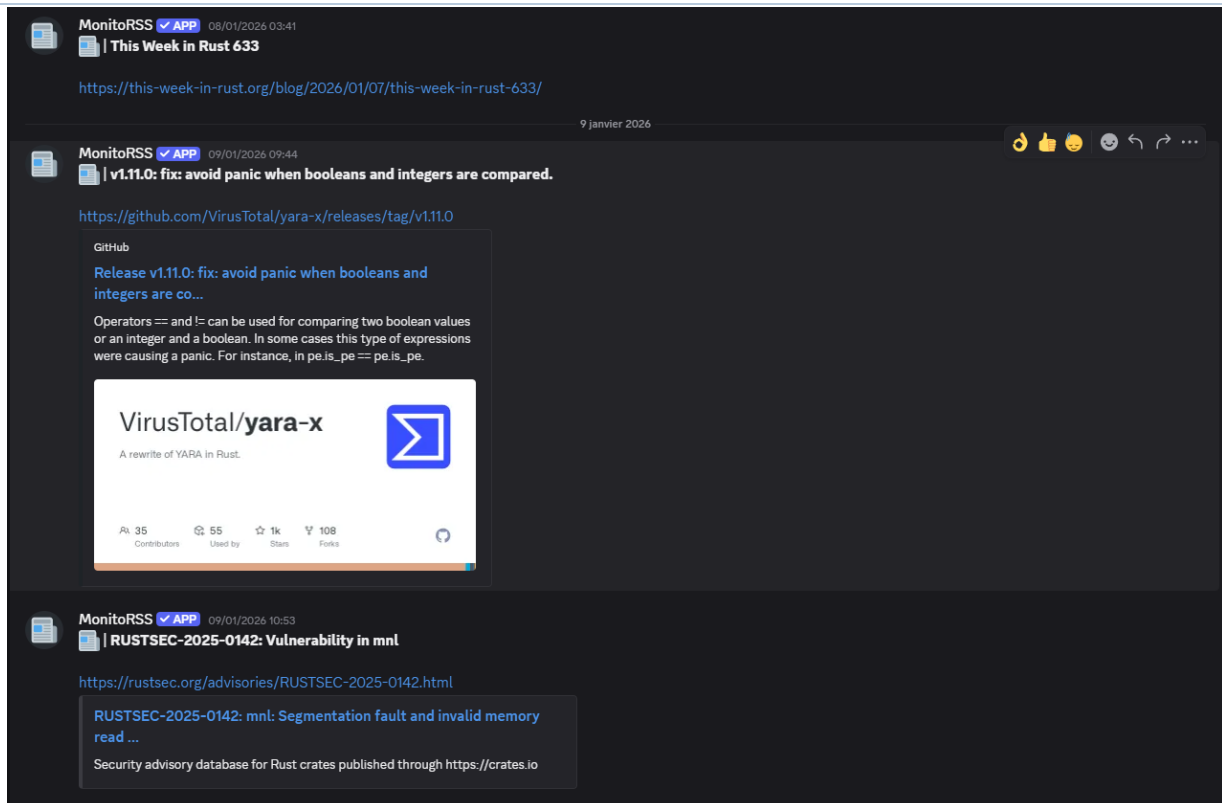
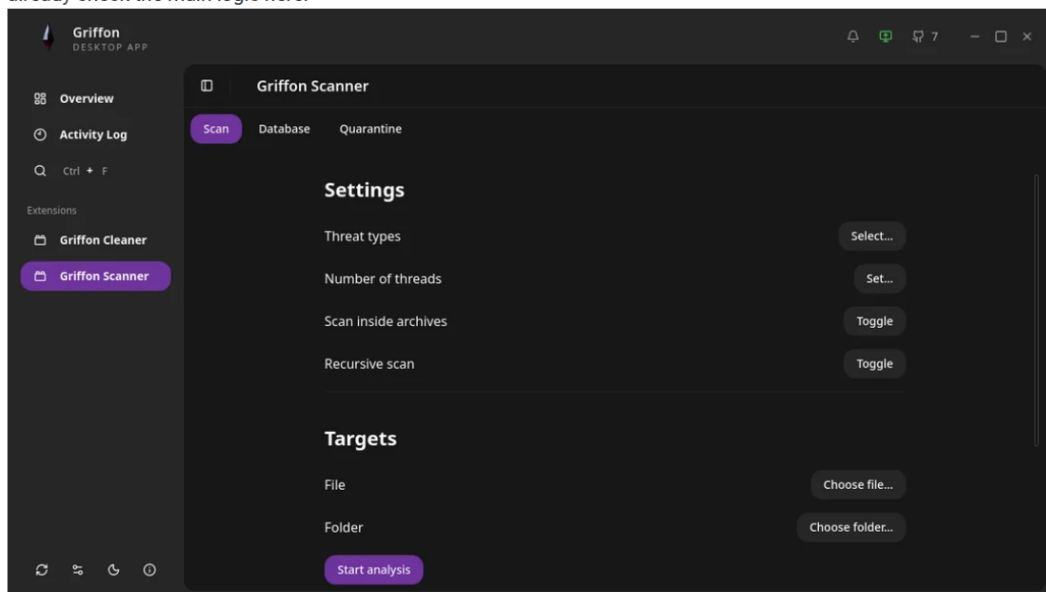
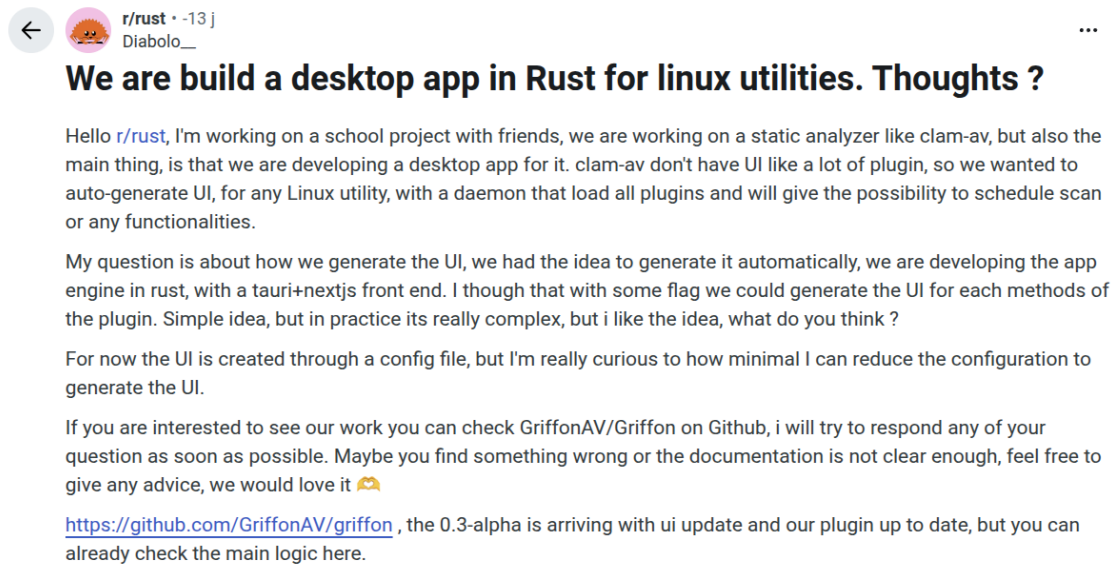


Figure 10 — Continuité de la veille : This Week in Rust, YARA-X et RUSTSEC en janvier 2026.

12.4.2 Sollicitation de la communauté Rust sur Reddit

Un membre de l'équipe a publié sur r/rust une présentation de Griffon et une question ouverte sur l'auto-génération de l'interface graphique à partir des plugins. La publication expose le besoin : daemon chargé de découvrir les plugins, application desktop Rust/Tauri et réduction de la configuration nécessaire pour générer la GUI. Elle fournit également un lien vers le dépôt GriffonAV/griffon et invite les lecteurs à signaler les erreurs, les limites de la documentation et les choix discutables.

La capture montre quatre commentaires, ce qui constitue une première interaction avec une communauté technique externe. Même si le dossier ne dispose pas encore d'une synthèse détaillée de chaque réponse, cette publication dépasse la simple diffusion promotionnelle : elle formule une question d'architecture précise et sollicite des retours sur une difficulté réelle du projet. Elle est donc recevable comme preuve de participation et d'ouverture, tout en restant une pratique à poursuivre et à mieux archiver.



↑ 0 ↓ 4

Figure 11 — Publication r/rust sollicitant des retours sur l'architecture et la GUI auto-générée de Griffon.

12.4.3 Apport au KPI et limites

- La veille est régulière et outillée : plusieurs sources sont suivies sur la durée et diffusées automatiquement dans l'espace de travail de l'équipe.
- Les sources sont directement liées à Griffon : Rust, sécurité des crates, Linux et YARA-X.
- La publication Reddit expose une décision structurante à une communauté spécialisée et demande explicitement des retours.
- Les captures fournissent une preuve datée et vérifiable de l'activité de veille et d'ouverture.
- La démarche reste encore ponctuelle côté contribution externe : les commentaires reçus, les décisions prises après échange et d'éventuelles contributions à des projets tiers devront être archivés plus systématiquement.

13. Synthèse d'impact et décisions

13.1 Impact sur la performance

- Rapports structurés et dashboards comparatifs disponibles.
- Amélioration exacte de 8,3 % sur deux runs safe comparables fournis.

- Amélioration agrégée visuelle estimée proche de 29 %, à confirmer par SQL.
- Réduction du volume des réponses Cleaner grâce à l'agrégation.
- Protocole binaire et payload borné pour le canal runner.

13.2 Impact sur la sécurité et la robustesse

- Rust pour le daemon et les plugins, avec contrat ABI explicite.
- YARA-X réduit la dépendance à une intégration C directe.
- Validation du magic, de la version, du type et de la taille des messages.
- Runner et heartbeat pour détecter les connexions défaillantes.
- Gestion explicite des erreurs et permissions dans le Cleaner.
- Packaging avec groupe Griffon afin de limiter l'usage de sudo pour les clients.

13.3 Impact sur la maintenabilité

- Architecture modulaire et workspace avec crates partagées.
- Contrat de plugin minimal et documentation de référence.
- UI déclarative réutilisable entre plugins.
- Issues découpées par responsabilité et PR revues.
- Conventions de commits facilitant le changelog et l'analyse historique.

13.4 Impact sur le temps de développement

L'investissement initial dans le Plugin Manager, l'ABI, l'IPC et le moteur d'interface augmente la complexité du socle. En contrepartie, chaque nouveau plugin peut réutiliser le template, l'interface partagée, le transport, la GUI et le packaging. Le coût marginal d'une nouvelle fonctionnalité diminue lorsque le contrat est stable.

13.5 Registre des décisions

Décision	Statut	Justification	Réévaluation
`abi_stable` pour les plugins	Adoptée	Meilleur compromis Rust-to-Rust et contrôle au chargement.	Si besoin multilangage/sandbox.
Frame + CBOR pour PM <-> runner	Adoptée	Messages typés, bornés et corrélables.	Lors du protocole v2.
YARA-X	Adoptée	API Rust et cohérence sécurité/maintenance.	À chaque mise à jour majeure.
Tauri	Adoptée	Réutilisation frontend web et backend Rust.	Si contraintes WebView ou packaging.
Grafana + SQLite	Adoptée pour reporting	Dashboards rapides et requêtes SQL.	Si volume ou automatisation CI augmente.
WebAssembly Component Model	Différée	Isolation prometteuse mais coût de migration trop élevé.	Après stabilisation du SDK plugin.

14. Matrice de conformité au KPI

Exigence	Éléments de preuve	État	Action résiduelle
----------	--------------------	------	-------------------

Exigence	Éléments de preuve	État	Action résiduelle
Veille régulière sur technologies du projet	Chronologie GitHub, docs YARA/IPC/Grafana, journal consolidé, flux RSS et relais MonitoRSS dans Discord.	Couvert	Conserver un export périodique des abonnements et des alertes ayant conduit à une action.
Sources variées et crédibles	Docs officielles Rust, RFC, YARA-X, Tauri, Grafana ; GitHub ; docs internes.	Couvert	Conserver une bibliographie mise à jour.
Tendances utiles au projet	ABI stable, CBOR, YARA-X, Tauri, WASM, observabilité.	Couvert	Rester centré sur les décisions projet.
Notes comparatives	Matrices ABI, formats, détection et GUI.	Couvert	Publier une version courte dans le wiki.
Benchmark technique	JSON versionné, SQLite, Grafana, comparaison v0.0.9/v0.1.0.	Couvert	Exporter les agrégats exacts et le protocole de machine.
Décisions raisonnées	Registre des décisions et alternatives rejetées/différées.	Couvert	Associer chaque décision à une ADR.
1–2 POC concrets	Plugin Manager, IPC v1, migration YARA-X.	Couvert	Joindre liens/commits dans l'annexe finale.
Intégration d'une technologie	`abi_stable`, CBOR, YARA-X, Tauri et Grafana intégrés.	Couvert	Aucune.
Mise à jour code/docs/architecture	Guides, manifests, protocoles, packaging, releases.	Couvert	Harmoniser Vite/Next.js et sources de règles.
Participation communautaire	Dépôt public, appels à contribution et publication r/rust sollicitant des avis sur l'architecture et la GUI auto-générée.	Couvert	Poursuivre les échanges et archiver les commentaires, réponses et décisions associées.
Partage dans l'équipe	PR reviews, branches, conventions, rapports et wiki.	Couvert	Archiver les comptes rendus de review.
Synthèse d'impact	Performance, sécurité, qualité, maintenance et temps de dev.	Couvert	Mettre à jour à chaque release.

Conclusion de conformité — Le dossier couvre l'ensemble des dimensions du KPI avec des preuves d'expérimentation, d'intégration, de mesure, de documentation, de veille régulière et d'échange communautaire. Les actions restantes concernent principalement la reproductibilité fine des agrégats Grafana et la capitalisation des retours externes dans la durée.

15. Limites, risques et feuille de route

15.1 Limites actuelles

- Le contrat de plugin repose encore sur des chaînes et réponses JSON pour les fonctions métier.

- Le manifeste peut être incohérent avec la bibliothèque sans validation automatique complète.
- La sandbox dépend principalement de la séparation de processus ; une `.so` native reste puissante.
- La provenance et la licence des règles YARA ne sont pas encore complètement documentées.
- Le benchmark ne fournit pas encore toutes les répétitions ni l'environnement matériel.
- Le chiffre agrégé du dashboard n'est pas accompagné de l'export SQL source.
- Certaines pages de documentation présentent des informations de stack divergentes.
- La participation communautaire reste ponctuelle : les captures RSS et Reddit prouvent la démarche, mais les réponses reçues et les décisions qui en découlent doivent encore être archivées plus systématiquement.

15.2 Priorités techniques

Priorité	Action	Bénéfice attendu
P0	Créer un validateur <code>.toml` <-> `init() ` <-> `handle_message() `</code> .	Détecter les erreurs avant installation.
P0	Ajouter pagination/limite métier aux listes de résultats.	Prévenir le retour du problème des 10 000 chemins.
P0	Exporter automatiquement les agrégats de benchmark.	Rendre les chiffres auditable.
P1	Définir une politique de versionnement du protocole et de l'ABI.	Évoluer sans casser les plugins.
P1	Signer les archives de plugins et vérifier l'intégrité.	Réduire le risque supply chain.
P1	Documenter et tester la base de règles YARA-X.	Maîtriser licences et faux positifs.
P2	POC WebAssembly Component Model/WASI.	Évaluer isolation et plugins multilingages.
P2	Ajouter fuzzing du parser Frame/CBOR.	Tester les entrées invalides et limites.

15.3 POC futur proposé : plugin WebAssembly

Le Component Model WebAssembly fournit une architecture pour des composants interopérables et des contrats WIT. Il pourrait permettre à Griffon de limiter les capacités d'un plugin et d'ouvrir la plateforme à d'autres langages. [EXT-WASM]

Élément	Proposition
Durée	Une journée de POC, sans migration du scanner.
Fonction test	Plugin <code>`ping`</code> ou calcul de hash, sans accès arbitraire au système.
Mesures	Temps de démarrage, latence d'appel, taille, mémoire, complexité du SDK.
Question	Le gain d'isolation compense-t-il le runtime et la redéfinition des capacités ?
Critère d'adoption	Contrat simple, overhead acceptable et politique de permissions claire.

16. Conclusion

Le KPI 1 est illustré par une transformation technique complète de Griffon. Le projet est passé d'un prototype monolithique à une plateforme modulaire distribuée sous forme de daemon, GUI, CLI et plugins. Cette évolution repose sur des recherches ciblées et des choix argumentés : ``abi_stable`` pour le contrat binaire, CBOR et un framing versionné pour le canal du Plugin Manager, YARA-X pour l'intégration du moteur de détection, Tauri pour l'application desktop et Grafana pour la comparaison des performances.

Les preuves ne se limitent pas à des intentions. Les pull requests retracent les différentes étapes de l'architecture ; les guides expliquent comment créer, charger et appeler un plugin ; le protocole est spécifié octet par octet ; les rapports du Cleaner contiennent des métriques réelles ; le dashboard permet de comparer les versions ; et un incident de production a conduit à limiter et agréger les données renvoyées.

La décision la plus importante n'est pas l'adoption d'une technologie isolée, mais la mise en place d'une méthode : questionner, comparer, expérimenter, mesurer, documenter et intégrer. Cette méthode réduit le risque de suivre une tendance sans bénéfice et permet à l'équipe de justifier aussi bien les choix adoptés que les choix différés, comme WebAssembly.

Avant la soutenance, deux actions renforceront encore la preuve : joindre l'export SQL exact des agrégats Grafana et documenter précisément l'environnement matériel des benchmarks. Les captures RSS/Discord et la publication Reddit couvrent désormais les critères de veille régulière et de participation communautaire.

Annexe A — Catalogue des preuves internes

Référence	Document	Utilisation
INT-README	README — Griffon Modular Security Platform for Linux	Présentation, installation, structure du dépôt et commandes.
INT-INTRO	Introduction — Welcome to Griffon	Positionnement, daemon, plugins, Scanner et Cleaner.
INT-INSTALL	Installation & Releases	Versions, compilation et exécution depuis les sources.
INT-YARA	YARA	Choix YARA-X et structure des règles.
INT-IPC	Griffon IPC Protocol (Frame + CBOR)	Format, types, limites et sécurité.
INT-GRAFANA	Grafana in This Project	Usage de Grafana et SQLite pour les comparaisons.
INT-POC	POC Template	Méthode de cadrage des expérimentations.
INT-PERF-TPL	PERF Report Template	Structure des rapports de performance.
INT-PLUGIN-ARCH	Building a Griffon Plugin	Modèle <code>.so` + `.toml`</code> .
INT-UI	The UI Manifest & Store	Déclaration du state et du layout.
INT-INTERACTIONS	Interactions	Connexion UI, store et backend.
INT-RUST-BACKEND	The Rust Backend	Contrat ABI et dispatch.
INT-PACKAGING	Packaging & Loading	Build, zip et installation.
INT-PLUGIN-GUIDE	Griffon Plugin Creation Guide	Interface partagée et checklist.
INT-REPORT	How to write a report	Nommage et structure des rapports.
INT-PR	Pull request validation workflow	Règles de revue et de fusion.
INT-COMMITS	Conventional Commit Messages	Types et formats de commits.
INT-BRANCH	Branch management	Stratégie main/develop/feature/hotfix.
INT-RSS-LIST	Capture — liste des flux RSS suivis	Preuve des abonnements Rust Foundation, The Hacker News et This Week in Rust.
INT-RSS-DISCORD-DEC	Capture — relais Discord de décembre 2025	Avis RUSTSEC, Rust dans le noyau Linux et release Rust 1.92.0.
INT-RSS-DISCORD-JAN	Capture — relais Discord de janvier 2026	This Week in Rust 633, release YARA-X v1.11.0 et avis RUSTSEC.
INT-REDDIT	Capture — publication sur r/rust	Sollicitation de retours externes sur l'architecture de plugins et la GUI auto-générée.

Annexe B — Liens GitHub significatifs

- [#90 — Plugin Manager v1](#)
- [#91 — IPC protocol v1](#)
- [#118 — daemon nouvelle architecture](#)
- [#122 — daemon fonctionnel](#)
- [#149 — packaging et groupe Griffon](#)
- [#175 — notifications et historique daemon](#)
- [#180 — notifications GUI](#)
- [Issues Cleaner](#)
- [Issue #130 — Scanner YARA-X](#)
- [Dépôt Griffon](#)

Annexe C — Exemple de rapport JSON

```
{
  "scenario": "basic_clean",
  "version": "v0.1.0",
  "mode": "safe",
  "plugin": "griffon_cleaner",
  "timestamp": "2026-07-05T12:50:05.414211791+00:00",
  "bytes_freed_total": 14291222939,
  "files_scanned_total": 110388,
  "files_cleaned_total": 0,
  "run_duration_seconds": 1.135,
  "bytes_freed_per_second": 12591385849.339207,
  "errors_by_type": {
    "permission_denied": 15
  }
}
```

Annexe D — Trame d'un rapport de veille futur

Champ	Contenu attendu
Identifiant	AAAA-MM-JJ_VEIL_NNN_sujet
Question	Problème Griffon auquel la veille doit répondre.
Sources	Documents officiels, dépôt, article/paper et discussion communautaire.
Alternatives	Au moins deux solutions comparables.
Hypothèse	Résultat attendu et critère d'échec.
POC	Branche, durée, protocole et limites.
Résultat	Mesures, captures, code et difficultés.
Décision	Adopté, rejeté, différé ou POC complémentaire.
Impact	Performance, sécurité, maintenance et délai.
Suivi	Issue, PR, documentation et date de réévaluation.

Annexe E — Conseils pour la soutenance

1. Commencer par le problème du prototype monolithique, pas par une liste de technologies.
2. Montrer le schéma daemon -> Plugin Manager -> runner -> plugins.
3. Expliquer en une phrase pourquoi l'ABI Rust native n'est pas suffisante.
4. Présenter la matrice `abi\_stable` / C ABI / WebAssembly et la décision.
5. Montrer le header IPC et la limite de 1 Mo.
6. Raconter l'incident des 10 000 chemins comme retour d'expérience principal.
7. Afficher Grafana et annoncer d'abord le résultat exact +8,3 % ; présenter +29 % comme tendance à confirmer.
8. Montrer une PR et un guide de plugin pour prouver l'intégration et le partage.
9. Terminer par les limites : validation de manifest, signature des plugins, benchmark reproductible et suivi dans le temps des retours communautaires.

Annexe F — Bibliographie externe

[EXT-RUST-ABI] Rust Reference — External blocks / ABI — <https://doc.rust-lang.org/reference/items/external-blocks.html> (consulté le 15 juillet 2026)

[EXT-ABI-STABLE] Documentation `abi\_stable` — https://docs.rs/abi_stable/ (consulté le 15 juillet 2026)

[EXT-CBOR] RFC 8949 — Concise Binary Object Representation — <https://www.rfc-editor.org/rfc/rfc8949.html> (consulté le 15 juillet 2026)

[EXT-YARAX-START] YARA-X — Getting started — <https://virustotal.github.io/yara-x/docs/intro/getting-started/> (consulté le 15 juillet 2026)

[EXT-YARAX-RUST] YARA-X — API Rust — <https://virustotal.github.io/yara-x/docs/api/rust/> (consulté le 15 juillet 2026)

[EXT-YARAX-STABLE] YARA-X is stable — annonce 1.0 — <https://virustotal.github.io/yara-x/blog/yara-x-is-stable/> (consulté le 15 juillet 2026)

[EXT-TAURI-ARCH] Tauri Architecture — <https://v2.tauri.app/concept/architecture/> (consulté le 15 juillet 2026)

[EXT-TAURI-SEC] Tauri Security — <https://v2.tauri.app/security/> (consulté le 15 juillet 2026)

[EXT-YAURI-CALL] Calling Rust from the Frontend — <https://v2.tauri.app/develop/calling-rust/> (consulté le 15 juillet 2026)

[EXT-GRAFANA-DASH] Grafana Dashboard JSON model — <https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/view-dashboard-json-model/> (consulté le 15 juillet 2026)

[EXT-WASM] WebAssembly Component Model — <https://component-model.bytecodealliance.org/> (consulté le 15 juillet 2026)

[EXT-SYSTEMD] systemd.service — <https://www.freedesktop.org/software/systemd/man/systemd.service.html> (consulté le 15 juillet 2026)

Annexe G — Pièces complémentaires restant à ajouter

Pièce	Pourquoi	Priorité
Export CSV/SQL du dashboard Grafana	Valider précisément les moyennes et variances.	Haute
Capture de la machine/protocole de benchmark	Rendre le test reproductible.	Haute
Capture de la GUI installant un plugin ZIP	Prouver le parcours utilisateur complet.	Moyenne
Capture des logs de handshake et d'appel IPC	Illustrer le fonctionnement réel du protocole.	Moyenne
Liste des sources/licences des règles YARA	Sécuriser la supply chain et la conformité.	Haute